

project Documentation: Academia AI Chatbot

1. Overview

Academia AI is a premium, RAG-powered (Retrieval-Augmented Generation) college FAQ assistant. It combines a modern, glassmorphic frontend with an intelligent AI backend (Hugging Face / Groq) to provide context-aware answers based on a local knowledge base.

2. Core Features

■ Premium UI/UX

- **Glassmorphism Design**: High-end interface with backdrop blur, smooth gradients, and custom scrollbars.
- **Dynamic Animations**: Fade-in messages and a pulsating "thinking" indicator to simulate human-like interaction.
- **Category Quick Buttons**: Instant navigation chips for Admissions, Hostel, Courses, etc.
- **Responsive Layout**: Optimized for both mobile and desktop views.

■ Intelligent RAG Engine

- **Retrieval Logic**: A weighted word-overlap algorithm caches and retrieves relevant FAQ snippets from the knowledge base.
- **AI Reasoning**: Uses the `openai/gpt-oss-20b:groq` model via the Hugging Face Router API.
- **Context Injection**: Each query is enriched with relevant knowledge to prevent AI hallucinations.
- **Soft Fallback**: Intelligent handling of missing information that guides users to general official channels.

■ Knowledge Base & Tracking

- **50+ FAQ Entries**: Structured JSON data covering all major college departments.
- **Local Feedback System**: 🟢/🔴 sentiment tracking stored in `localStorage` for analysis.
- **Query Logging**: Background logging of unanswered queries to identify knowledge gaps.

3. Technology Stack

- **Frontend**: React (Vite)
- **Styling**: Vanilla CSS (CSS Variables, Flexbox/Grid, Glassmorphism)
- **API Connectivity**: Hugging Face Router (Groq)
- **Environment Management**: Vite `.env``

4. Logical Flow & Architecture

A. User sends a query

The query is handled in `ChatInterface.jsx``. The UI enters an `isTyping`` state.

B. Context Retrieval

`intentHandler.js`` scans `faqData.json`` using a scoring algorithm:

1. **Direct Keyword Match**: +3 points per match.
2. **Word Overlap (Question + Keywords)**: +1 point per matching word.
3. **Selection**: Top 4 matches are compiled into a "Context Snippet."

C. AI Generation

`aiService.js`` sends the query and context to Hugging Face:

- **System Prompt**: Instructs the AI to prioritize the context and stay professional.
- **Auth**: Secured via `VITE_HF_TOKEN``.

D. Display & Feedback

The UI renders the AI response using `MessageBubble.jsx``, which includes the feedback tracking system.

5. Output Formats

****Sample AI Request (Internal API Call)****

```
{
  "model": "openai/gpt-oss-20b:groq",
  "messages": [
    {
      "role": "system",
      "content": "You are Academia AI... KNOWLEDGE BASE CONTEXT: Q: What are the hostel fees?
A: Hostel fees are $1500-$2500..."
    },
    {
      "role": "user",
      "content": "how much for the dorms?"
    }
  ]
}
```

****Sample AI Response****

```
{
  "choices": [
    {
      "message": {
        "content": "Based on our official records, hostel fees (dorms) range from $1,500 to $2,500 per
semester depending on whether you choose a Single or Double room and AC/Non-AC options."
      }
    }
  ]
}
```

****Feedback Data (LocalStorage)****

```
[
  {
    "messageId": "1713181234567",
    "query": "Is there a gym?",
```

```
"status": "helpful",  
"timestamp": "2024-04-15T17:30:00Z"  
}  
]  
  
---
```

6. Project Structure

```
project/  
■■■■ .env # API Secrets  
■■■■ index.html # Meta/SEO  
  
■■■■ src/  
■ ■■■■ components/ # UI Elements  
■ ■■■■ data/ # FAQ JSON & PDF  
■ ■■■■ logic/ # AI Service & Context Engine  
■ ■■■■ App.jsx # Entry Point  
■ ■■■■ index.css # Global Styles  
■■■■ RAG_SETUP.md # Ingestion Guide
```